

Shop the Look: Visual Product Discovery

Technical document. Source code, UI and tests are in the GitHub repository; every number below comes from `app/final_eval.py` (validation), `tests/check.py` and `tests/edge_cases.py` run against the shipped server.

1. Overall approach

Given a scene photo, the system (1) detects each fashion item, (2) turns every item crop into an embedding, (3) searches the catalog by cosine similarity (the provided catalog, plus an optional extended catalog), (4) decides whether a result is an exact match or only a similar product, and (5) explains each recommendation from attributes (category, color, pattern, material, style). Nothing is trained from scratch: pretrained fashion vision-language models are used zero-shot, and the validation file is used to *measure and calibrate* the system.

```
scene image -> OWLv2 open-vocabulary detector (29 prompts, 25 labels) -> candidate boxes
box -> crop embedding (Marqo-FashionSigLIP) -> relabel -> verify (not clothing? looks like
its label?) -> declutter
item -> cosine search over 28,094 provided + 44,072 optional extended products
    -> rank = cosine + family + colour - hubness penalty; hide weak and duplicate matches
    -> exact-vs-similar threshold -> zero-shot attribute tags -> explanation sentence
FastAPI /api/search -> web UI (boxes over the scene, one card per item beside the photo,
Extended-catalog switch, three colour themes)
```

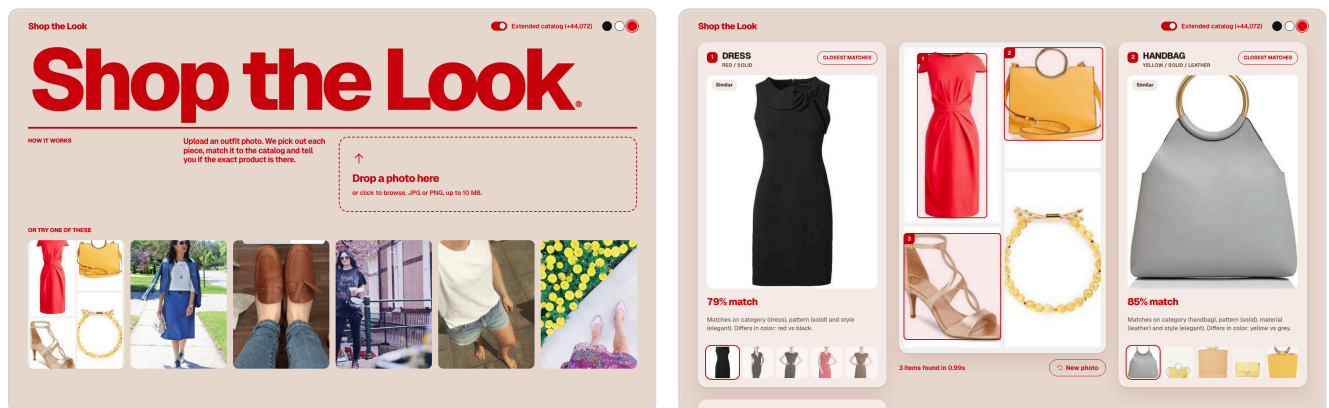


Figure 1. The demo UI. Left: upload screen with sample photos. Right: results, with the detected items outlined on the photo and one card per item (large top match, match percentage, explanation, and a strip of the other matches).

2. Data and pipeline facts

- **Catalog:** 50,000 rows containing product IDs only (no title, category or color). Only **28,094** IDs are unique; rows are de-duplicated (28 of the remaining images are blank and are blocked). All attributes shown in the UI therefore have to be inferred from the images.
- **Images:** scene and product images share the Pinterest URL scheme given in the brief. All **28,415** images (28,094 products + 321 validation scenes) downloaded with 64 parallel threads in **554 s** (about 51 images/s) with **0 failures**. A sequential download at roughly 1.2 s per image would take about 9 hours, so parallelism was essential.
- **Validation:** 330 (product, scene) pairs. For **288** pairs the product is in the catalog; for **42** it is not, which gives natural "no exact match exists" cases that are used to calibrate the exact-match threshold.
- **Observation that shaped the design:** catalog photos are product shots on models (a "top" photo also shows a skirt and a bag), and many ground-truth products are accessories (handbags, sunglasses, earrings), so the detector vocabulary was extended beyond clothing.

3. Model architecture and methodology

Detection and verification

OWLv2 (`owlv2-base-patch16-ensemble`) is prompted with 29 text queries covering 25 labels (tops, dresses, jackets, pants, skirts, shoes, boots, sneakers, heels, handbags, sunglasses, earrings, necklaces, belts, ties, suits, blazers and others; synonyms such as "boots" and "leather belt" feed one label).

Detection is split into four stages so each can be tuned without re-running the detector:

1. **Candidates:** score cut-off 0.2 (footwear 0.1; belts 0.2); garments scoring 0.1 to 0.2 (a half-cropped shirt or trousers) are kept as "weak" and survive only if the crop looks at least 70% like that garment type; boxes must be at least 6% of the long side in one direction and 1.2% in the other (thin belts survive, specks do not), left and right shoe merged into one item, greedy NMS (IoU 0.5).
2. **Relabel:** every crop is embedded once and zero-shot classified. If another garment family is at least 40% likely and at least 3x likelier than the label's own, the label is corrected, but only between tops, bottoms, dresses and outerwear (wide-leg trousers detected as "suit" become bottoms; a short-sleeve shirt detected as "coat" becomes a shirt). Within a family, a wrong word is also corrected ("skirt" to "pants") when the label's word is under 20% and the best word at least 60%.
3. **Verify:** belts need stronger evidence than other items, because waistbands, hems and hands look like belts: of the 66 belt candidates in the validation scenes only about 8 were real belts (all scored 0.64 or more), the rest 0.35 to 0.60. A belt is kept only if the detector score is at least 0.62, or the crop looks like a belt at least 60% with score at least 0.4. Otherwise a crop is dropped if it looks like non-clothing (car, light streaks, wall, bare skin, face and 15 more classes; skin and hands are ignored for watches, bracelets, necklaces and other worn accessories, where they are normal) with probability of at least 0.5, or if it does not look like its own label and the detector is not confident.
4. **Declutter:** a top, skirt or jacket box inside a whole-dress or whole-suit box is part of that outfit piece, and a box inside a bigger box of the same kind (handbag strap inside handbag) is a duplicate; both are removed. If nothing survives, the whole image is used only when it clearly is one clothing item (flat-lay product photo); otherwise the app reports "no clothing found".

Embedding model

Marqo-FashionSigLIP (ViT-B/16, 512-d, L2-normalised) embeds both crops and catalog images, and its text tower produces zero-shot attribute tags. It embeds both the provided and the extended catalog. It was compared with FashionCLIP on the validation set and won (section 6).

Product representation

Each catalog product is one 512-d vector of its full image, plus a zero-shot *garment-family* probability vector (for extended products the real product type from their metadata is used instead) (15 families: tops, bottoms, dresses, outerwear, shoes, bags, eyewear, hats, scarves, belts, ties, bracelets, necklaces, earrings, watches) derived from the same embedding. I also tried multi-view representations (full, top half, bottom half, center crop) to counter the "several items per product photo" problem; they **lowered** recall (pooled R@10 from 8.3% to 1.4%) because every extra view adds look-alike false positives, so the final index is single-view. This is a measured negative result, not an assumption.

4. Retrieval and ranking

- **Search:** exact cosine similarity as one matrix product, at most 72,166 x 512 floats, milliseconds. No approximate index is needed (FAISS also segfaulted next to PyTorch on this macOS setup).
- **Ranking:** $\text{score} = \text{cosine} + 0.2 \times \text{family} + 0.1 \times \text{colour} + 0.1 \times \text{sleeve} - 0.5 \times \text{hubness}$. *Family* is the zero-shot garment-family probability of the product (94% accurate against the 37,155 extended products that have a real type label; bracelets 98%, belts 99%, ties 99.6%); extended

products use their true type. Families are fine-grained (belts, ties, bracelets, necklaces, watches, hats are separate) so a belt query cannot be answered with ties; watches and bracelets share a group because detectors call a wrist item either. *Colour* uses a colour-similarity table (red is close to pink, navy to blue, silver to grey) applied when the crop's colour tag is at least 50% confident. At the earlier weight of 0.05 only 88.7% of shown top-5 results matched the photographed item's colour (a red dress returned a black one); at 0.1, 96.5% do, for a top-100 recall cost of about 3 points because true pairs in this dataset often differ in colour. *Sleeve* (short, long, sleeveless; tops, outerwear and dresses only) is a zero-shot tag used in the same way, so a short-sleeve shirt does not return long-sleeve coats; it is neutral on validation, whose labels are too loose to reward it. Weights were chosen on the validation pairs (grid in `tune_rank.py`, `tune_hub.py`).

- **Hubness correction:** a few products sit near every query. Measured on 1,223 test crops, a single provided-catalog product filled 5.7% of all top-5 slots and the top 20 products filled 21%. Each product is penalised by its mean similarity to its 10 nearest typical query crops (a CSLS-style correction, `hub.npy`). At weight 0.5 top-100 recall rose from 31.6% to 35.1% on the extended index while the top-20 share of result slots fell from 10% to 6.3%; weights above 0.75 hurt, so 0.5 is used.
- **Colour-first display:** when the crop's colour is at least 60% confident, products of a clearly different colour (similarity under 0.5) only fill slots left after all right-colour candidates, and the shown match percentage is the visual similarity minus a colour and sleeve disagreement penalty, so order and percentage always agree and a black dress cannot be listed as the closest match to a red one (an earlier version re-sorted by raw similarity and did exactly that). The raw cosine alone decides the "Exact match" badge. Cost on validation: top-5 recall 10.4% (11.5% without the rule) on the provided catalog and 9.4% (10.1%) with the extended catalog; top-100 falls from 37.5% to 28.8% and from 32.3% to 26.7%, which only affects results beyond the five shown.
- **Display rules:** top 5 per item, near-identical catalog photos (cosine above 0.985, the same product listed twice) collapsed to one, matches below cosine 0.5 hidden (this hides about 10% of true validation matches because true pairs are often loosely related; 0.55 would hide 22%), and an item with no remaining match is not shown at all.
- **Catalog hygiene (6,737 products blocked):** blank images (28 in the provided catalog, 21 extended), an "image coming soon" placeholder, and extended products that are not look items (fragrance, makeup, socks, innerwear, wallets, rings, cufflinks, gift sets, gloves, umbrellas, shoe laces) never appear in results (`data/blocked.json`). Before wallets were blocked, 925 wallets could be returned for handbag queries.
- **Exact vs similar:** "Exact match" only if the raw cosine is at least **0.882**, the 95th percentile of the best cosine for the 42 validation products known to be absent from the catalog (false "exact" claims measured at 7.1%).

Explanations

Zero-shot tags are computed for the crop and for each match by softmax over text prompts ("a {color} colored garment", "a garment made of {material}" and so on). The explanation lists the attributes that agree and those that differ, e.g. *"Matches on category (skirt), color (denim blue) and material (denim). Differs in pattern: solid vs floral."* Near-synonym categories (top, blouse, t-shirt) count as one family. These are template sentences grounded in measured agreement, so they cannot invent attributes the models did not see. An attribute is stated only when both the crop and the match tag it with at least 40% softmax confidence, and pattern, material and style are never stated for eyewear and jewelry, where those tags are unreliable.

5. Evaluation methodology

For every validation pair whose product is in the catalog (288), the scene runs through the full pipeline (detect, relabel, verify, declutter, rank). The pair's rank is the best rank of the true product over the scene's

items, and counts only if the true product's cosine is above the display floor (otherwise the UI hides it). This matches the UI, which shows a top-K list per item. Reported: recall@K and MRR against a random baseline of K/N, and the share of pairs whose true garment family was detected.

System (288 pairs)	R@5	R@10	R@100
FashionCLIP, whole scene as query (pooled, 28,094 products)	1.7%	2.4%	10.1%
FashionCLIP + OWLv2 crops (pooled)	3.5%	4.2%	16.0%
FashionSigLIP + crops (pooled)	6.2%	8.3%	18.8%
FashionSigLIP + crops + 4-view product index (pooled)	1.0%	1.4%	7.6%
Detection fixes + family bonus, per-item top-K	11.8%	14.6%	34.7%
Ranking with verification, family, colour, sleeve, hubness, floor (before the colour-first display rule), provided catalog	11.5%	17.4%	37.5%
The same, Extended catalog on (72,166 candidates)	10.1%	15.6%	32.3%
Shipped system (colour-first display), provided catalog only	10.4%	15.3%	28.8%
Shipped system, Extended catalog on (default)	9.4%	13.9%	26.7%
Random ranking (28,094 / 72,166 products)	0.02% / 0.007%	0.04% / 0.014%	0.36% / 0.14%

Shipped, provided catalog: R@1 4.9%, R@50 25.7%, MRR 0.076 (extended: R@50 21.5%, MRR 0.072); the right garment family was among the detected items for 87.5% of pairs. The first four rows pool all crops into one ranking, which is stricter than the shipped per-item view, so compare rows only within the same metric. Latency is about 0.6 s per photo on an Apple M-series GPU and 1.4 s on its CPU.

Robustness tests and deployment check (tests/)

Recall on validation pairs does not show how the system fails on bad input, so two automated suites run against the live API. **Image suite** (`check.py` , 68 images, an explicit expected result per group): three motion-blurred photos, six crops of real failure screenshots (half-cropped short-sleeve shirt, trousers with no belt, red dress with yellow bag), real photos from earlier failures (shoes, belt, suit, a car), seven non-clothing images, and 48 half-cropped garments (top, bottom and left half of T-shirts, shirts, trousers, track pants, tracksuits, jeans). **68 of 68 pass**; non-clothing images returned 7 invented items before verification and 0 after; the true product is in the top 5 for 31 of 48 half or full garment images. The suite found two real bugs after the headline numbers looked fine: half-visible garments were rejected by a score cut-off that the crop check can safely override, and an upload error card appeared off-screen. **Edge-case suite** (`edge_cases.py` , 12 inputs): empty, corrupt and truncated files get 400; tiny, grey, transparent, panorama, WebP, GIF and BMP images are handled; over 10 MB gets 413; a 30,000 x 30,000 image gets 413 (it used to crash the server with a 500); six simultaneous requests return identical results. **Clean-install check**: a new virtual environment built only from `requirements.txt` , forced to CPU, passed both suites at about 1.4 s per photo. The Dockerfile is untested (Docker's service was not running). The blurred and crop test images were made from screenshots of the app, so they contain its box overlays and are indicative only.

6. Extended catalog (dataset expansion) and accuracy work

The provided catalog is the company's inventory, so the expansion is **optional and clearly separated**. An **Extended catalog** switch in the UI (on by default) adds **44,072** products from the public Fashion Product Images dataset (Hugging Face mirror `benitomartin/fashion-product-images-small-384x512` , 384x512 photos with name, type and color). Extended results carry an "Extended" badge and their product name, and the switch turns them off to see only the provided catalog. Check the dataset's license before any commercial use.

- **Speed:** the five 2 GB parquet shards downloaded in parallel in about 5 minutes; unpacking took 3 s (5 processes); embedding all 44,072 images with the same model and single full-image view took 800 s. End to end, about 20 minutes.
- **Cleaning:** a pixel-variance scan found **28 blank images in the provided catalog** and 21 in the extended set (one blank image scored 82 to 88% against unrelated items because it sits near the centre of the embedding space), and a placeholder reading "image coming soon". All are blocked from results. 6,690 extended products that are not look items (fragrance, makeup, skincare, socks, innerwear, wallets, rings, cufflinks, gift sets, gloves, umbrellas, shoe laces) are excluded, 6,737 blocked products in total with the blank and placeholder images.
- **Effect on accuracy (measured):** with 72,166 candidates (2.6x more distractors), recall on the validation pairs stays close: top-5 9.4% (10.4% on the provided catalog alone), top-10 13.9% (15.3%), top-100 26.7% (28.8%). Validation labels refer to products in the provided catalog only, so the extension cannot raise this number; its benefit is more choice per detected item and real product names. The exact-match threshold was recalibrated for the larger index and came out the same (0.882).

Other accuracy work: choosing the embedding model on validation (FashionSigLIP over FashionCLIP, about 2x R@10); adding accessory, footwear, suit and tie prompts after inspecting failures; the garment-family prior; detector noise filtering; and testing multi-view products (rejected).

7. Limitations and future improvements

- **Weak labels:** validation pairs are loosely matched (the paired product often resembles the scene item without being the identical item), so recall values understate real usefulness; user-level relevance judgments would be a better target.
- **No catalog metadata:** color, material and style come from zero-shot image tags and can be wrong (for example "denim" on a white shirt in one test). A VLM captioning pass over the catalog would give cleaner attributes.
- **Multi-item catalog photos:** a product photo of a top also contains a skirt and a bag. Detecting and cropping the main item of each catalog image (about 6 hours with OWLv2 on this laptop at 0.8 s per image, far less on a server GPU) is the most promising fix.
- **Fine-tuning:** a contrastive adapter trained on scene-crop to product pairs (with a held-out split) should lift recall; with 288 usable pairs only a light linear adapter is safe.
- **Look-alike products:** small shoe crops can match flip-flops photographed as bare footbeds, because the ranking separates shoes from accessories but not heels from flat sandals. A word-level (heels vs sneakers) bonus was tested and lowered validation recall (top-5 10.1% to 9.4% at the smallest weight), so it is not used.
- **Detection:** heavily blurred photos can return nothing (the app then says no clothing was found); small or occluded items and multi-person scenes are the weak cases; thresholds are per class but hand-tuned on a small set, so unusual photos may need retuning. Sharpness could not be used to warn about blur: scores of rescaled screenshots (180 to 500) overlap those of sharp crops (225 to 340).
- **Scale:** exact search is fine to about a million items; beyond that use an ANN index (HNSW) and pre-computed attribute tags.